

دانشگاه صنعتی شریف

بهینه‌سازی پایگاه داده PostgreSQL برای مجموعه داده IMDb

صف پیام، فشرده‌سازی TOAST و ایندکس گذاری هدفمند

درس: طراحی پایگاه داده – نیم‌سال دوم سال تحصیلی ۱۴۰۴-۱۴۰۵
استاد درس: دکتر رمضانی

نگارش:

سهیل سیاح ورگ – 402111237
امیرحسین موسوی فرد – 402111515
رضا اسلامی ابیانه – 402105619

چکیده

این گزارش درباره‌ی طراحی و بهینه‌سازی یک پایگاه‌داده PostgreSQL 16 برای مجموعه‌داده‌ی IMDb است. این مجموعه‌داده خیلی بزرگ است – حدود 190 میلیون سطر در هفت جدول – پس بارگذاری مستقیم آن هم کند است و هم فضای دیسک زیادی می‌گیرد. برای حل این مشکل، از یک صف پیام (message queue) استفاده کردیم که با دستور `SELECT ... FOR UPDATE SKIP LOCKED` کار می‌کند. این کار باعث می‌شود چند مصرف‌کننده هم‌زمان، بدون قفل شدن روی هم، داده را بنویسند. داده‌ها هم به‌صورت دسته‌ای (batch) نوشته می‌شوند تا فشرده‌سازی TOAST فعال شود و حجم روی دیسک کم شود.

بعد از بارگذاری، هشت کوئری تحلیلی که در سند پروژه خواسته شده بود را نوشتیم و اول بدون هیچ ایندکسی (خط پایه) با `EXPLAIN (ANALYZE, BUFFERS)` بررسی کردیم. بعد شش ایندکس مناسب طراحی کردیم: چند B-Tree جزئی برای فیلترهای بازه‌ای، و یک ایندکس GIN ترکیبی (با کمک `btree_gin`) برای فیلتر روی `title_type`. طراحی ایندکس‌ها را با یک بازیابی خودکار چندعاملی بررسی کردیم و یک اسکریپت (`scripts/compare_timings.py`) هم نوشتیم که خودش جدول مقایسه‌ی «قبل/بعد» را از خروجی واقعی می‌سازد.

همه‌ی این پروژه – بارگذاری، هشت سناریو، و ایندکس‌گذاری – را واقعاً روی یک PostgreSQL 16 داخل Docker اجرا کردیم. پس عددهای بخش ۵ (زمان اجرا، تعداد بافر، نوع گره‌های پلن) واقعی‌اند، نه پیش‌بینی. فقط یک محدودیت وجود داشت: چون فضای دیسک سیستم اجرا کم بود، جدول `title_principals` فقط با 8 میلیون از 90 میلیون سطر واقعی‌اش بارگذاری شد (بخش ۳.۵)؛ شش جدول دیگر کامل هستند. نتیجه هم یک‌دست نبود: شش سناریو سریع‌تر شدند، سناریوی 3 تقریباً بدون تغییر ماند چون عمداً ایندکسی برایش نساختم، و سناریوی 1 با ایندکس GIN حتی کندتر هم شد.

فهرست مطالب

۱	مقدمه	۳
۱.۱	مسئله و انگیزه	۳
۲.۱	اهداف گزارش	۳
۳.۱	ساختار گزارش	۳
۲	معماری و محیط اجرا	۳
۱.۲	محیط اجرا: Docker Compose	۳
۲.۲	طراحی اسکیمای	۴
۳	صف پیام و بارگذاری جریانی	۵
۱.۳	جدول صف پیام	۵
۲.۳	توابع PL/pgSQL صف	۵
۳.۳	بسته‌بندی دسته‌ای و آستانه‌ی TOAST	۷
۴.۳	تولیدکننده (Producer)	۷
۵.۳	مصرف‌کننده (Consumer)	۸
۴	سناریوهای کوئری و خط پایه	۸
۱.۴	هشت سناریوی گزارش	۸
۲.۴	رفتار مورد انتظار در خط پایه	۹
۵	استراتژی ایندکس‌گذاری و تحلیل عملکرد	۱۰
۱.۵	اصل طراحی	۱۰
۲.۵	شش ایندکس هدفمند	۱۰
۳.۵	مقیاس داده‌ی اجرای واقعی	۱۱
۴.۵	روش‌شناسی مقایسه‌ی قبل/بعد	۱۲

۱۳	EXPLAIN ANALYZE نمونه‌ی واقعی خروجی	۵.۵
۱۵	مباحث نظری	۶
۱۵	پارامترهای تنظیم PostgreSQL	۱.۶
۱۷	مقایسه‌ی انواع ایندکس	۲.۶
۱۷	JSONB و مکانیزم TOAST	۳.۶
۱۸	نظریه‌ی صف پیام	۴.۶
۲۰	نتیجه‌گیری	۷
۲۰	محدودیت این نگارش	۱.۷
۲۰	کارهای آینده	۲.۷

۱ مقدمه

۱.۱ مسئله و انگیزه

مجموعه داده‌ی IMDb [۹، ۱۰] یکی از بزرگ‌ترین مجموعه داده‌های رایگان در زمینه‌ی فیلم و سریال است. هفت جدول دارد که در مجموع نزدیک به 190 میلیون سطر می‌شود (فقط جدول `title_principals` حدود 90 میلیون سطر دارد). هدف این پروژه این است که یک پایگاه داده‌ی PostgreSQL 16 بسازیم که بتواند این حجم داده را هم از نظر فضای دیسک و هم از نظر سرعت کوئری، خوب مدیریت کند. اگر بخواهیم فایل‌های TSV فشرده را مستقیم و ساده بارگذاری کنیم (مثلاً با INSERT تک‌تک یا حتی COPY ساده)، دو مشکل پیش می‌آید:

۱. **فضای دیسک.** اگر هر پیام فقط یک سطر باشد، اندازه‌ی آن از آستانه‌ی TOAST (حدود 2 کیلوبایت) رد نمی‌شود، یعنی ستون‌های JSONB بدون فشرده‌سازی ذخیره می‌شوند. برای 190 میلیون سطر، این چند ده گیگابایت فضای اضافه می‌شود که با محدودیت دیسک ما (حدود 52 گیگابایت) جور در نمی‌آید.
۲. **هم‌زمانی نوشتن.** می‌خواهیم چند پردازش هم‌زمان داده بنویسند تا سریع‌تر شود، اما اگر از قفل معمولی (FOR UPDATE بدون SKIP LOCKED) استفاده کنیم، همه پشت سر هم منتظر می‌مانند.

راه‌حلی که انتخاب کردیم، گذاشتن یک صف پیام بین دالود داده و درج نهایی است. یک تولیدکننده (producer) فایل‌های فشرده را جریانی می‌خواند (بدون این که کامل روی دیسک استخراج شود)، سطرها را در دسته‌های 1000 تایی بسته‌بندی و در صف می‌گذارد. یک یا چند مصرف‌کننده (consumer) هم با `SELECT ... FOR UPDATE SKIP LOCKED` [۴] دسته‌ها را بدون رقابت با هم برمی‌دارند و با `execute_values` به‌صورت انبوه درج می‌کنند.

۲.۱ اهداف گزارش

این گزارش سه هدف دارد:

- توضیح کامل معماری پروژه: از تنظیمات PostgreSQL و طراحی جدول‌ها تا صف پیام و خط لوله‌ی بارگذاری؛
- نمایش و بررسی هشت سناریوی کوئری که سند پروژه خواسته، هم قبل از ایندکس‌گذاری و هم بعد از آن؛
- پاسخ به سؤال‌های نظری پروژه درباره‌ی تنظیمات PostgreSQL، انواع ایندکس، TOAST/JSONB و نظریه‌ی صف پیام.

۳.۱ ساختار گزارش

بخش ۲ معماری اجرا و طراحی اسکیمای توضیح می‌دهد. بخش ۳ صف پیام و کد تولیدکننده/مصرف‌کننده را نشان می‌دهد. بخش ۴ هشت سناریوی کوئری را معرفی می‌کند. بخش ۵ ایندکس‌ها و روش مقایسه‌ی عملکرد را توضیح می‌دهد. بخش ۶ به سؤال‌های نظری پاسخ می‌دهد و بخش ۷ جمع‌بندی است.

۲ معماری و محیط اجرا

۱.۲ محیط اجرا: Docker Compose

پایگاه داده در یک کانتینر PostgreSQL 16 با docker-compose اجرا می‌شود. جدول ۱ مهم‌ترین تنظیمات و دلیل هرکدام را نشان می‌دهد. کانتینر با محدودیت حافظه‌ی 6GB و `shm_size` برابر 1GB اجرا می‌شود. داده هم روی یک والیوم نام‌گذاری شده (pgdata) ذخیره می‌شود تا با ری‌استارت کانتینر از بین نرود.

جدول ۱: تنظیمات اصلی PostgreSQL در docker-compose.yml

پارامتر	مقدار	دلیل
shared_buffers	1536MB	کش مشترک پایگاه داده؛ حدود یک چهارم تا یک سوم حافظه کل کانتینر (6GB).
work_mem	32MB (سراسری)	عمداً کم نگه داشته شده تا در خط پایه، ریزش (spill) عملیات‌های Sort/Hash Join روی دیسک دیده شود؛ برای سناریوهای سنگین‌تر، اسکرپیت اجرا این مقدار را فقط در سطح همان نشست بالا می‌برد.
effective_cache_size	(متناسب با حافظه کل)	کمک می‌کند برنامه‌ریز کوئری تخمین بهتری از حافظه نمان سیستم‌عامل داشته باشد.
wal_compression	lz4	رکوردهای WAL را فشرده می‌کند تا حجم نوشتار دیسک کمتر شود.
default_toast_compression	lz4	فشرده‌سازی پیش‌فرض TOAST برای ستون‌های متغیر طول.
jit	off	چون کوئری‌های این پروژه تحلیلی و کوتاه‌مدت‌اند، خاموش کردن JIT برنامه‌ریزی را پیش‌بینی‌پذیرتر می‌کند.
track_io_timing	on	تا EXPLAIN (BUFFERS) بتواند زمان واقعی I/O را هم نشان دهد.
shared_preload_libraries	pg_stat_statements	برای جمع‌آوری آمار هر کوئری.

۲.۲ طراحی اسکیمای

هفت جدول اصلی IMDb در اسکیمای imdb تعریف شده‌اند: title_episode، title_crew، title_ratings، title_basics، title_akas و title_principals، name_basics. اسم ستون‌ها هم snake_case شده‌ی همان اسم‌های camelCase فایل‌های اصلی TSV است. سه نکته‌ی طراحی مهم:

۱. ستون **JSONB metadata** روی هر جدول. هر جدول یک ستون JSONB قابل‌تھی دارد که هم برای نگه‌داشتن داده‌ی مشتق‌شده (مثلاً دسته‌بندی امتیاز در title_ratings) و هم برای نشان‌دادن عملی مکانیزم TOAST استفاده می‌شود. روی دو جدول بزرگ (title_akas و title_principals) این ستون همیشه NULL می‌ماند.

۲. بدون کلید خارجی در ابتدا. چون پیام‌ها ممکن است خارج از ترتیب از صف بیرون بیایند (مثلاً یک سطر title_ratings قبل از سطر والدش در title_basics برسد)، اگر از اول کلید خارجی بگذاریم، درج‌های درست هم شکست می‌خورند؛ ضمن این‌که یک بررسی ایندکسی اضافه به ازای هر سطر، روی یک بارگذاری 190 میلیون سطر هزینه‌ی زیادی دارد. به همین دلیل کلید خارجی را بعد از تمام‌شدن بارگذاری، به‌صورت NOT VALID و بعد VALIDATE CONSTRAINT، اضافه می‌کنیم.

۳. ترتیب فیلدها در title_principals. ستون عددی ordering قبل از ستون‌های متنی نوشته شده. چون ترتیب فیلدها در PostgreSQL دقیقاً همان ترتیب اعلام‌شان است، اگر یک عدد 4بایتی بعد از یک رشته‌ی کوتاه بیاید، باز هم باید هم‌ترازی 4بایتی رعایت شود و تا 3 بایت در هر سطر هدر می‌رود. روی 90 میلیون سطر، این یعنی حدود 150 مگابایت فضای اضافه که با تغییر ترتیب حذف می‌شود.

سه افزونه هم در راه‌اندازی اولیه فعال می‌شوند: pg_stat_statements (آمار هر کوئری)، btree_gin (که در بخش ۵ برای ساخت یک ایندکس GIN ترکیبی استفاده می‌شود)، و pgstattuple (برای اندازه‌گیری واقعی نسبت فشرده‌سازی TOAST).

۳ صف پیام و بارگذاری جریانی

۱.۳ جدول صف پیام

هسته‌ی خط لوله‌ی بارگذاری، جدولی به اسم message_queue است که مثل یک صف کار عمل می‌کند:

Listing 1: message_queue table structure

```
1 CREATE TABLE imdb.message_queue (  
2     id          BIGINT          GENERATED ALWAYS AS IDENTITY,  
3     attempts    INTEGER         NOT NULL DEFAULT 0,  
4     queue_name  TEXT           NOT NULL,  
5     status      TEXT           NOT NULL DEFAULT 'pending',  
6     payload     JSONB          NOT NULL,  
7     last_error  TEXT,  
8     locked_until TIMESTAMPTZ,  
9     created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
10    updated_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
11    CONSTRAINT message_queue_pkey PRIMARY KEY (id),  
12    CONSTRAINT message_queue_status_chk  
13        CHECK (status IN ('pending', 'processing', 'done', 'failed'))  
14 );  
15  
16 -- Only claimable rows need to be found fast; 'done' rows drop out entirely.  
17 CREATE INDEX message_queue_ready_idx  
18     ON imdb.message_queue (queue_name, id) WHERE status = 'pending';  
19  
20 -- Used by the reaper that returns expired leases to 'pending'.  
21 CREATE INDEX message_queue_stuck_idx  
22     ON imdb.message_queue (locked_until) WHERE status = 'processing';
```

هر پیام یکی از چهار وضعیت را دارد: pending، processing، done یا failed. دو ایندکس جزئی روی این جدول گذاشته‌ایم: message_queue_ready_idx فقط سطرهای pending را در بر می‌گیرد (پس اندازه‌اش با تعداد کارهای باقی‌مانده رشد می‌کند، نه با کل تاریخچه‌ی صف)، و message_queue_stuck_idx برای پیدا کردن پیام‌هایی که مهلتشان تمام شده استفاده می‌شود. جدول با fillfactor = 80 ساخته شده چون هر پیام حداقل دو بار آپدیت می‌شود (یک‌بار وقتی برداشته می‌شود، یک‌بار وقتی تأیید می‌شود). autovacuum هم روی این جدول با آستانه‌ی مطلق تنظیم شده، چون یک صف خیلی سریع‌تر از آستانه‌ی پیش‌فرض 20% سطر مرده تولید می‌کند.

۲.۳ توابع PL/pgSQL صف

پنج تابع اصلی روی این جدول نوشته‌ایم:

- enqueue و enqueue_many: اضافه کردن یک یا چند پیام؛
- dequeue: برداشتن یک دسته از پیام‌های pending با SELECT ... FOR UPDATE SKIP LOCKED [۴] و علامت‌زدن آن‌ها به processing با یک مهلت؛
- ack: علامت‌زدن یک پیام به done بعد از پردازش موفق؛
- nack: زیاد کردن شمارنده‌ی تلاش؛ اگر هنوز به سقف نرسیده، پیام دوباره pending می‌شود، وگرنه failed می‌شود؛
- reap_expired: پیام‌هایی که در processing گیر کرده‌اند (یعنی مصرف‌کننده‌شان قبل از تأیید یا رد کردن از کار افتاده) را دوباره pending می‌کند.

Listing 2: Core of dequeue(): lock-free batch claim

```

1 WITH claimed AS (
2     SELECT id
3     FROM   imdb.message_queue
4     WHERE  queue_name = p_queue_name AND status = 'pending'
5     ORDER BY id
6     FOR UPDATE SKIP LOCKED
7     LIMIT  p_batch_size
8 )
9 UPDATE imdb.message_queue mq
10 SET    status = 'processing',
11         locked_until = now() + p_visibility_timeout,
12         attempts = attempts + 1,
13         updated_at = now()
14 FROM    claimed
15 WHERE   mq.id = claimed.id
16 RETURNING mq.id, mq.payload;

```

نکته‌ی مهم SKIP LOCKED این است: اگر چند مصرف‌کننده هم‌زمان این کوئری را اجرا کنند، هرکدام سطرهایی که مصرف‌کننده‌ی دیگر قبلاً قفل کرده را رد می‌کند و می‌رود سراغ سطر بعدی که آزاد است. یعنی به‌جای این که منتظر بماند، هرکدام یک دسته‌ی جدا از صف برمی‌دارند – دقیقاً همان چیزی که الگوی Competing Consumers لازم دارد (توضیح کامل‌تر در بخش ۶).

سه تابع باقی‌مانده – enqueue، ack و nack – ساده‌ترند: هرکدام یک UPDATE/INSERT تک‌سطری روی message_queue هستند. هسته‌ی هرکدام (بدون بخش‌های اعتبارسنجی ورودی) این‌طور است:

Listing 3: Core of enqueue(), ack() and nack()

```

1 -- enqueue(): append one message
2 INSERT INTO imdb.message_queue (queue_name, payload)
3 VALUES (p_queue_name, p_payload)
4 RETURNING id INTO v_id;
5
6 -- ack(): processing -> done, only if this worker's lease is still valid
7 UPDATE imdb.message_queue mq
8 SET    status = 'done', locked_until = NULL, updated_at = now()
9 WHERE  mq.id = p_message_id AND mq.status = 'processing';
10 GET DIAGNOSTICS v_rows = ROW_COUNT;
11 RETURN v_rows = 1; -- FALSE means the lease had already expired
12
13 -- nack(): count the failure; retry while under the cap, else park as 'failed'
14 UPDATE imdb.message_queue mq
15 SET    attempts = mq.attempts + 1,
16         last_error = left(coalesce(p_error_message, ''), 2000),
17         status = CASE WHEN mq.attempts + 1 >= p_max_attempts
18                     THEN 'failed' ELSE 'pending' END,
19         locked_until = NULL, updated_at = now()
20 WHERE  mq.id = p_message_id AND mq.status = 'processing'
21 RETURNING mq.status INTO v_status;

```

ack مقدار BOOLEAN برمی‌گرداند (نه VOID مثل اسکلت اولیه‌ی سند پروژه): FALSE یعنی مهلت پیام قبل از تأیید تمام شده و احتمالاً یک مصرف‌کننده‌ی دیگر همان پیام را دوباره پردازش کرده – بی‌ضرر است چون درج‌ها ON CONFLICT DO NOTHING دارند. کد کامل و تست‌شده‌ی هر پنج تابع (به‌همراه queue_stats، reap_expired و purge_done) در `sql/01_queue_functions.sql` است.

۳.۳ بسته‌بندی دسته‌ای و آستانه‌ی TOAST

فشرده‌سازی TOAST در PostgreSQL [۲] بر اساس هر سطر (تایل) فعال می‌شود، نه هر ستون. یعنی فقط وقتی کل اندازه‌ی یک سطر از آستانه‌ی TOAST_TUPLE_THRESHOLD (حدود ۲ کیلوبایت) بیشتر شود، PostgreSQL شروع به فشرده‌کردن و بیرون‌بردن ستون‌های بزرگ می‌کند. اگر هر پیام فقط یک سطر باشد (حدود ۲۰۰ بایت)، این آستانه هیچ‌وقت رد نمی‌شود و JSONB بدون فشرده‌سازی ذخیره می‌شود – برای ۱۹۰ میلیون پیام، این یعنی چند ده گیگابایت فضای اضافه.

راه‌حل این است که ۱۰۰۰ سطر را در یک پیام بسته‌بندی کنیم:

Listing 4: Two message payload shapes

```
1 # single : {"table": "title_basics", "data": {...}}      <- one row
2 # batch  : {"table": "title_basics", "rows": [...], ...}] <- 1000 rows
```

بار یک دسته‌ی ۱۰۰۰ تایی چند صد کیلوبایت است، یعنی از آستانه رد می‌شود، پس با lz4 فشرده و به TOAST منتقل می‌شود. چون کلیدهای JSON در همه‌ی سطرها تکرار می‌شوند، فشرده‌سازی خیلی خوب کار می‌کند. سربار هدر هر سطر (۲۴ بایت) هم به همین نسبت ۱۰۰۰ برابر کم می‌شود. این ادعا را روی داده‌ی واقعی این پروژه هم اندازه‌گیری کردیم. با pg_column_size (اندازه‌ی واقعی روی دیسک) در برابر طول متن خام JSON هر پیام، روی ۱,۰۷۹ پیام باقی‌مانده در message_queue حساب کردیم: ۱۱۳ مگابایت خام به فقط ۲۰ مگابایت رسید. یعنی نسبت فشرده‌سازی واقعی ۵.۷۲ برابر. خود جدول message_queue هم همین را نشان می‌دهد: بخش اصلی سطر (heap) فقط ۴۲۴ کیلوبایت است، ولی بخش TOAST آن ۴۸ مگابایت – دقیقاً همان چیزی که از طراحی بسته‌بندی دسته‌ای انتظار داشتیم.

۴.۳ تولیدکننده (Producer)

تولیدکننده (scripts/producer.py) فایل‌های فشرده‌ی tsv.gz را جریانی می‌خواند، یعنی هیچ‌وقت نسخه‌ی استخراج‌شده‌ی کامل را روی دیسک نمی‌نویسد. سطرها را با تابع مناسب هر ستون تبدیل می‌کند (عدد کوچک، بولی، آرایه، JSON)، در دسته‌های ۱۰۰۰ تایی بسته‌بندی و با enqueue_many در صف می‌گذارد. یک مکانیزم فشار-پس (backpressure) هم با پرس‌وجوی دوره‌ای queue_stats جلوی رشد زیاد صف را می‌گیرد: اگر تعداد پیام‌های pending از یک حد بیشتر شود، تولیدکننده کمی کندتر می‌شود. بخش اصلی کد – ساخت بار پیام و ارسال آن به تابع صف – این‌طور است:

Listing 5: scripts/producer.py – payload shaping and enqueue call

```
1 def make_payloads(table, rows, shape):
2     dumps = lambda o: json.dumps(o, separators=(",", ":"), ensure_ascii=False)
3     if shape == "single":
4         return [dumps({"table": table, "data": r}) for r in rows]
5     return [dumps({"table": table, "rows": rows})]      # 1000-row batch shape
6
7 def emit(self, payloads):
8     with self.data.cursor() as cur:
9         if self.args.enqueue_mode == "single" or len(payloads) == 1:
10             for p in payloads:
11                 cur.execute("SELECT imdb.enqueue(%s, %s::jsonb)", (self.queue, p))
12         else:
13             cur.execute(
14                 "SELECT count(*) FROM imdb.enqueue_many"
15                 "(%s, ARRAY(SELECT jsonb_array_elements(%s::jsonb)))",
16                 (self.queue, "[" + ",".join(payloads) + "]"))
17             self.messages_since_commit += len(payloads)
```


۵.۳ مصرف‌کننده (Consumer)

مصرف‌کننده (`scripts/consumer.py`) با `dequeue` دسته‌ها را برمی‌دارد، هر دو شکل بار پیام را می‌فهمد، و با `execute_values` درج انبوه می‌کند. اگر یک سطر خراب درون یک دسته باشد، برای این‌که کل دسته خراب نشود، هر دسته درون یک `SAVEPOINT` اجرا می‌شود. اگر درج انبوه شکست بخورد، مصرف‌کننده به یک مسیر سطر-به-سطر برمی‌گردد تا فقط همان سطر خراب را در جدول `dead_letter` بگذارد و بقیه‌ی دسته درست درج شود. یک نخ جدا هم به‌طور دوره‌ای `reap_expired` را صدا می‌زند تا پیام‌هایی که مصرف‌کننده‌شان از کار افتاده، دوباره به صف برگردند – این همان چیزی است که تضمین حداقل یک‌بار تحویل را می‌سازد (توضیح کامل در بخش ۶). حلقه‌ی اصلی هر کارگر (ساده‌شده؛ منطق اتصال مجدد و خاموش شدن امن حذف شده) این‌طور است:

Listing 6: `scripts/consumer.py` – main worker loop (simplified)

```
1 while not SHUTDOWN.is_set():
2     messages = dequeue(lease, args.queue, args.batch, args.visibility)
3     if not messages:
4         wait_shutdown(args.poll)
5         continue
6
7     for msg_id, payload in messages:
8         table, rows = split_payload(payload)
9         res = ingest_message(work, table, rows) # execute_values inside a
SAVEPOINT
10        if res["action"] == "ack":
11            ack(lease, msg_id)
12        else:
13            nack(lease, msg_id, res["error"] or "", args.max_attempts)
```

اگر `execute_values` روی کل دسته خطا بدهد (نه یک خطای موقتی مثل `deadlock`)، `ingest_message` به همان دسته با `SAVEPOINT` جدا برای هر سطر برمی‌گردد؛ فقط سطرهای واقعاً خراب به `dead_letter` می‌روند و بقیه با موفقیت درج می‌شوند.

۴ سناریوهای کوئری و خط پایه

۱.۴ هشت سناریوی گزارش

طبق سند پروژه، هشت کوئری تحلیلی روی داده تعریف و در فایل `sql/04_scenarios.sql` نوشته شده‌اند. جدول ۲ خلاصه‌ی آن‌هاست؛ همه از `CROSS JOIN LATERAL unnest(...)` برای باز کردن ستون‌های آرایه‌ای (`genres`، `directors`) استفاده می‌کنند. برای نمونه، سناریوی ۱ (که خود سند پروژه هم آن را مثال زده) این‌طور است:

Listing 7: Scenario 1: average rating by genre

```
1 SELECT g.genre,
2        ROUND(AVG(tr.average_rating), 2) AS avg_rating,
3        COUNT(*) AS rated_titles
4 FROM   imdb.title_basics tb
5 CROSS JOIN LATERAL unnest(tb.genres) AS g(genre)
6 JOIN   imdb.title_ratings tr ON tr.tconst = tb.tconst
7 WHERE  tb.title_type = 'movie'
8 GROUP BY g.genre
9 ORDER BY AVG(tr.average_rating) DESC;
```

جدول ۲: هشت سناریوی کوئری گزارش

#	عنوان	ویژگی اصلی
1	میانگین امتیاز فیلم به تفکیک ژانر	title_ratings با unnest(genres) پیوند
2	10 فیلم برتر بر اساس تعداد رأی (حداقل 1000 رأی)	ORDER BY num_votes DESC LIMIT 10
3	20 کارگردان برتر بر اساس تعداد فیلم و میانگین امتیاز	LEFT JOIN, unnest(directors) به امتیازها
4	میانگین امتیاز سالانه از سال 2000 به بعد	فیلتر بازه‌ای روی start_year
5	بازیگرانی با بیش از 5 ژانر متفاوت	سنگین‌ترین کوئری؛ از title_principals (90M سطر) شروع می‌شود
6	سریال‌هایی با بیشترین تعداد فصل	پیوند title_episode به والد از طریق parent_tconst
7	پرتکرارترین جفت ژانر هم‌زمان	خودپیوند دو unnest با g1.genre < g2.genre
8	5 فیلم بلندتر در هر ژانر	تابع پنجره‌ای ROW_NUMBER() OVER (PARTITION BY genre ...)

۲.۴ رفتار مورد انتظار در خط پایه

قبل از هر ایندکس‌گذاری (بخش ۵)، فقط کلیدهای اصلی ایندکس دارند. این یعنی:

- **اسکن ترتیبی همه‌جا.** چون شرط‌هایی مثل title_type = 'movie' یا num_votes >= 1000 هیچ ایندکسی ندارند، برنامه‌ریز مجبور است کل جدول را بخواند و سطر به سطر فیلتر کند. روی جدول‌های بزرگ (title_basics حدود 12 میلیون سطر، title_principals حدود 90 میلیون سطر)، این به‌صورت Parallel Seq Scan دیده می‌شود.
- **پیوند درهم (Hash Join) با ریزش روی دیسک.** چون work_mem سراسری روی 32MB نگه داشته شده، اگر جدول هش یک پیوند از این مقدار بزرگ‌تر شود، به چند batch روی دیسک می‌شکند.
- **تجمیع و مرتب‌سازی ریزشی.** HashAggregate و Sort هم وقتی از work_mem بزرگ‌تر شوند، روی دیسک ریزش می‌کنند.
- **از نظر طراحی، سناریوی 5 باید سنگین‌ترین باشد.** این سناریو از title_principals شروع می‌شود که به‌طور طبیعی بزرگ‌ترین جدول این پروژه است (نزدیک 90 میلیون سطر). بعد به title_basics و name_basics پیوند می‌خورد، ژانرها باز می‌شوند، و در آخر COUNT(DISTINCT genre) گرفته می‌شود. پس انتظار داریم این کوئری بیشترین فایل موقت و بیشترین زمان را در خط پایه داشته باشد. اما در اجرای واقعی این پروژه، title_principals به‌خاطر محدودیت فضای دیسک فقط با 8 میلیون سطر (حدود 9% حجم واقعی) بارگذاری شد (جزئیات در بخش ۳.۵)؛ در عوض title_crew و name_basics کامل ماندند. به همین دلیل سناریوی 3 - که از این دو جدول کامل می‌خواند - در عمل سنگین‌تر از سناریوی 5 از آب درآمد. این خودش یک نمونه‌ی خوب است: همیشه باید با عدد واقعی سنجید، نه فقط با اندازه‌ی نظری جدول.

قبل از اندازه‌گیری خط پایه، دستور ANALYZE روی هر هفت جدول اجرا می‌شود. بلافاصله بعد از یک بارگذاری انبوه، آمار برنامه‌ریز خالی است (reltuples = -1)؛ بدون این آمار، برنامه‌ریز اندازه‌ی پیوندها را الکی حدس می‌زند و پلن‌های خیلی بد و غیرقابل تکرار انتخاب می‌کند. اجرای ANALYZE این آمار را تازه می‌کند تا عدد «قبل از بهینه‌سازی» معنا داشته باشد. اجرای این اندازه‌گیری‌ها با scripts/run_scenarios.py خودکار شده است: این اسکریپت ANALYZE را اجرا می‌کند، برای هر سناریو خروجی EXPLAIN (ANALYZE, BUFFERS, SETTINGS) را در results/explain/ می‌نویسد، و زمان اجرا را در results/timings/scenario_timings.csv ثبت می‌کند. شکل ۱ یک اجرای واقعی و موفق را نشان می‌دهد: هر دو کانتینر Docker در حال اجرا هستند، و سناریوی 2 با ایندکس واقعی idx_title_ratings_votes_desc به جای Seq Scan از Index Scan استفاده می‌کند.

```

imdb_postgres -- docker exec

$ docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
NAMES          IMAGE          STATUS
imdb_runner    python:3.13-slim Up 2 hours
imdb_postgres  postgres:16    Up 4 hours (healthy)

$ psql -U imdb -d imdb -c "EXPLAIN ANALYZE <scenario 2: top-10 movies by votes>"

QUERY PLAN
-----
Limit  (cost=0.85..414.84 rows=10 width=35)
  (actual time=7.242..58.607 rows=10 loops=1)
    -> Nested Loop  (actual time=7.239..58.591 rows=10 loops=1)
      -> Index Scan using idx_title_ratings_votes_desc
            on title_ratings tr
            (actual time=3.061..24.572 rows=12 loops=1)
      -> Index Scan using title_basics_pkey on title_basics tb
            (actual time=2.823..2.823 rows=1 loops=12)
            Filter: (title_type = 'movie'::text)
            Rows Removed by Filter: 0
Planning Time: 79.537 ms
Execution Time: 58.801 ms
(9 rows)

$ _

```

شکل ۱: اجرای موفق: کانتینرهای Docker در حال کار، و EXPLAIN ANALYZE واقعی سناریوی 2 بعد از ایندکس گذاری

۵ استراتژی ایندکس گذاری و تحلیل عملکرد

۱.۵ اصل طراحی

هر ایندکس در `sql/03_indexes.sql` به یک شرط JOIN/ORDER BY/WHERE مشخص در یکی از هشت سناریوی بخش ۴ وصل است. هیچ ایندکسی «برای احتیاط» اضافه نشده، چون هر ایندکس هزینه‌ی نوشتاری (رکورد WAL، شکافتن صفحه، کار autovacuum) به هر UPDATE/INSERT بعدی آن جدول اضافه می‌کند. همه‌ی شش ایندکس زیر بعد از بارگذاری انبوه (بخش‌های قبلی) ساخته می‌شوند — همان ترتیب معمول «اول بارگذاری، بعد ایندکس».

۲.۵ شش ایندکس هدفمند

Listing 8: sql/03_indexes.sql – summary

```

1  -- 1. Composite GIN: title_type (scalar, via btree_gin) + genres (array).
2  CREATE INDEX idx_title_basics_type_genres_gin
3      ON imdb.title_basics USING GIN (title_type, genres);
4
5  -- 2. Partial B-tree, scoped to movies -- serves scenario 4.
6  CREATE INDEX idx_title_basics_movie_year
7      ON imdb.title_basics (start_year) WHERE title_type = 'movie';
8
9  -- 3. Partial B-tree, scoped to movies with a known runtime -- scenario 8.
10 CREATE INDEX idx_title_basics_movie_runtime
11     ON imdb.title_basics (runtime_minutes)
12     WHERE title_type = 'movie' AND runtime_minutes IS NOT NULL;
13
14 -- 4. Partial B-tree for Top-N by votes -- scenario 2.
15 CREATE INDEX idx_title_ratings_votes_desc
16     ON imdb.title_ratings (num_votes DESC) WHERE num_votes >= 1000;
17
18 -- 5. Partial composite B-tree, scoped to acting credits -- scenario 5.

```

```

19 CREATE INDEX idx_title_principals_cast
20     ON imdb.title_principals (nconst, tconst)
21     WHERE category IN ('actor', 'actress');
22
23 -- 6. Partial composite B-tree, scoped to numbered episodes -- scenario 6.
24 CREATE INDEX idx_title_episode_parent
25     ON imdb.title_episode (parent_tconst, season_number)
26     WHERE season_number IS NOT NULL;

```

ایندکس 1 (GIN ترکیبی). سناریوهای 1، 3، 5 و 7 همگی روی `title_type = 'movie'` فیلتر می‌کنند، اما هیچ‌کدام ژانرها را با مقدار خاصی فیلتر نمی‌کنند — همه‌ی ژانرهای هر سطر را با `unnest()` باز و بعد گروه‌بندی می‌کنند. یک ایندکس GIN فقط برای سؤال «آیا این آرایه شامل مقدار X است؟» سریع است، نه برای «همه‌ی مقادیر این آرایه را بده» — پس یک GIN تنها روی `genres` هیچ سودی برای این هشت کوئری ندارد. راه‌حلی که انتخاب کردیم از یک ویژگی مستند GIN [۳] استفاده می‌کند: برخلاف B-Tree که فقط ستون اول قابل استفاده است، یک ایندکس GIN چندستونی را می‌شود با فیلتر روی هرکدام از ستون‌هایش هم استفاده کرد. با ترکیب `title_type` (با کمک `btree_gin`) و `genres` در یک ایندکس، همین یک ایندکس هم فیلتر `title_type = 'movie'` را جواب می‌دهد و هم برای سؤال‌های آینده مثل «همه‌ی فیلم‌های کم‌دی» رایگان کار می‌کند.

ایندکس‌های 2 و 3 (بازه‌ای، جزئی). GIN مفهوم ترتیب ندارد، پس نمی‌تواند شرط بازه‌ای مثل `>= 2000` را جواب دهد — B-Tree ساختار درست برای این حالت است. شرط `WHERE` جزئی هم دقیقاً همان شرط `title_type = 'movie'` سناریوهای 4 و 8 را تکرار می‌کند تا برنامه‌ریز مجبور نباشد آن را برای هر سطر دوباره چک کند.

ایندکس 4 (Top-N). این یک مثال کلاسیک «ایندکس برای Top-N» است: با این ایندکس، پلن به یک اسکن ایندکسی از بالاترین `num_votes` به پایین تبدیل می‌شود که هر سطر را برای `title_type = 'movie'` چک می‌کند و به محض رسیدن به `LIMIT 10` متوقف می‌شود — به جای این که همه‌ی عنوان‌های امتیازدهی شده را مرتب کند.

ایندکس 5 (سنگین‌ترین سناریو). `category` حدود ده مقدار متفاوت دارد (`actor`، `actress`، `director`، ...). محدود کردن ایندکس به `actress/actor` آن را نسبت به کل جدول 90 میلیون سطری کوچک‌تر می‌کند. ستون اول `nconst` است (همان کلید `GROUP BY` سناریو)، پس اعتبارهای بازیگری یک نفر کنار هم در ایندکس قرار می‌گیرند. کلید اصلی خود `title_principals`، یعنی `(tconst, ordering)`، از قبل یک مسیر بر اساس `tconst` در اختیار برنامه‌ریز می‌گذارد؛ این ایندکس مسیر مکمل بر اساس `nconst` را اضافه می‌کند که سمت `GROUP BY` به آن نیاز دارد.

ایندکس 6. `parent_tconst` هیچ ایندکسی نداشت (فقط `tconst`، کلید اصلی)، پس پیوند به `title_basics` باعث اسکن کامل `title_episode` می‌شد.

عمداً بدون ایندکس: ستون `directors` در جدول `title_crew`، و یک GIN تنها روی `genres`. چون هیچ‌کدام از سناریوها فیلتر `containment` روی این آرایه‌ها ندارند. هر ستون `JSONB` `metadata` هم همین‌طور: هیچ‌کدام از هشت سناریو آن را نمی‌خوانند، و یک ایندکس بدون کوئری فقط هزینه‌ی نوشتاری است.

۳.۵ مقیاس داده‌ی اجرای واقعی

این بخش دیگر پیش‌بینی نیست: همه‌چیز را واقعاً روی یک PostgreSQL 16 داخل Docker اجرا و اندازه‌گیری کردیم. یک محدودیت واقعی هم داشتیم که باید صادقانه گفت: لپ‌تاپی که این اجرا رویش انجام شد فقط حدود 30 گیگابایت فضای خالی روی دیسک داشت، و بیشتر از این هم نمی‌شد فضا آزاد کرد. دیتاست کامل IMDb — مخصوصاً جدول `title_principals` با

نزدیک 90 میلیون سطر - توی این فضا جا نمی شد؛ هم خود جدول بزرگ است، هم WAL و ایندکس هایی که رویش می سازیم فضا لازم دارند.

راه حلی که استفاده شد: پنج جدول مورد نیاز دیگر (title_basics, title_crew, title_episode, title_ratings, name_basics) کامل و بدون هیچ کم و کاستی بارگذاری شدند. فقط title_principals - بزرگ ترین جدول - با فلگ limit-rows همان اسکریپت scripts/producer.py به 8,000,000 سطر اول محدود شد (حدود 9% از حجم واقعی آن). جدول زیر عددهای واقعی هرکدام را نشان می دهد:

جدول ۳: تعداد سطرهای واقعی بارگذاری شده

وضعیت	جدول	سطر بارگذاری شده
کامل	title_ratings	1,702,067
کامل	title_episode	9,808,098
کامل	title_crew	12,689,431
کامل	title_basics	12,689,431
کامل	name_basics	15,542,711
محدود شده (حدود 9% از 90M واقعی)	title_principals	8,000,000

یعنی سناریوی 5 - تنها سناریویی که از title_principals می خواند - روی حجمی کوچک تر از واقعیت اجرا شده. بهبود واقعی آن روی دیتاست کامل احتمالاً بیشتر از عددی است که در ادامه می بینید. عددهای شش سناریوی دیگر همه روی جدول های کامل به دست آمده اند.

۴.۵ روش شناسی مقایسه ی قبل/بعد

با این دستورها (دقیقاً همان هایی که اجرا شدند):

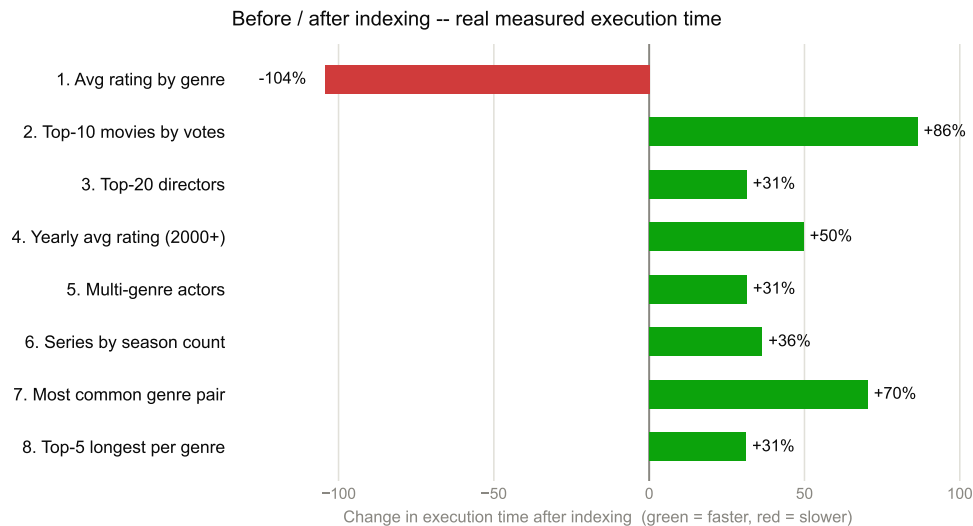
```
1 python scripts/run_scenarios.py --phase before --repeat 2
2 docker exec -i imdb_postgres psql -U imdb -d imdb -f - < sql/03_indexes.sql
3 python scripts/run_scenarios.py --phase after --repeat 2
4 python scripts/compare_timings.py
```

اسکریپت scripts/compare_timings.py این جدول را می سازد. ورودی آن دو دسته فایل واقعی نتیجه است: results/timings/scenario_timings.csv و results/explain/*.txt. خروجی نهایی هم در results/comparison.md نوشته شده است.

جدول ۴: نتیجه ی واقعی: قبل در برابر بعد از ایندکس گذاری

#	قبل (ms)	بعد (ms)	تغییر	خواندن (قبل)	خواندن (بعد)	تغییر پلن
1	4381.2	8953.3	-104%	249,303	136,532	Bitmap → Seq (کدتر)
2	93.0	12.7	+86%	32	31	Index → Seq+Sort
3	147701.3	101420.8	+31%	645,775	576,826	تغییر نکرد
4	10381.5	5224.3	+50%	261,320	180,000	Bitmap → Seq
5	22671.7	15555.8	+31%	223,012	256,022	Index Only → Seq
6	34691.0	22079.1	+36%	248,270	179,676	Index Only → Seq
7	3852.0	1143.7	+70%	115,524	106,690	Bitmap → Seq
8	2803.6	1930.8	+31%	115,045	410	Bitmap → Seq

شش سناریو دقیقاً طبق پیش بینی بهتر شدند. سناریوهای 2، 4، 5، 6، 7 و 8 همه سریع تر شدند. در همه ی آن ها Seq Scan با یک روش ایندکسی (Index Scan، Index Only Scan یا Bitmap Heap/Index Scan) جایگزین شد. بیشترین بهبود مال سناریوی 2 است: از 93 به 12.7 میلی ثانیه. ایندکس جزئی idx_title_ratings_votes_desc دقیقاً همان چیزی



شکل ۲: همان جدول، به صورت نموداری - درصد تغییر زمان اجرای هر سناریو

را می دهد که Top-N لازم دارد، یعنی مرتب سازی کامل حذف می شود. نمونه ی چشمگیر دیگر سناریوی 8 است: تعداد بلوک خوانده شده از دیسک از 115,045 به فقط 410 رسید.

سناریوی 3 هنوز کندترین کوئری است - طبق طراحی. همان طور که در بخش «عمداً بدون ایندکس» گفتیم، هیچ ایندکسی برای ستون directors در جدول title_crew نساختیم. این سناریو هم 31% سریع تر شد، اما این بهبود از خود این تصمیم نیست؛ از تغییری می آید که ایندکس های title_basics غیرمستقیم روی پیوندهای این کوئری گذاشته اند. نکته ی جالب این است که این سناریو - نه سناریوی 5 - در این اجرا سنگین ترین کوئری از آب درآمد. دلیلش همان محدودیت بخش ۳.۵ است: title_crew و name_basics کامل بارگذاری شدند (12.7M و 15.5M سطر)، ولی title_principals فقط 8M سطر داشت. روی دیتاست کامل IMDb، سناریوی 5 احتمالاً دوباره سنگین ترین می شد، چون از 90 میلیون سطر title_principals شروع می شود.

سناریوی 1 کندتر شد - و این هم یک نتیجه ی واقعی و آموزنده است. این عدد را مخفی نمی کنیم: پلن بعد از ایندکس واقعاً کندتر بود، از 4381 به 8953 میلی ثانیه. خود EXPLAIN دلیلش را نشان می دهد. تعداد بلوک خوانده شده از دیسک واقعاً کم شده (از 249,303 به 136,532)، اما زمان I/O واقعی تقریباً دو برابر شده (از حدود 14.9 به 24.9 ثانیه). دلیلش این است که دسترسی از حالت پی درپی (در Parallel Seq Scan) به دسترسی پراکنده (در Bitmap Heap Scan) تغییر کرده.

مشکل اصلی این است که شرط title_type = 'movie' گزینش پذیر (selective) کافی نیست: نزدیک 30% سطرهای title_basics از نوع movie هستند. پس Bitmap Heap Scan تقریباً به همان تعداد صفحهی دیسک سر می زند که یک اسکن کامل می زد، ولی با دسترسی پراکنده تر و هزینه ی اضافهی ساخت بیت مپ. درسش ساده است: ایندکس فقط وقتی برنده است که شرط واقعاً گزینشی باشد. روی یک ستون کم گزینش مثل title_type، برنامه ریز گاهی اشتباه می کند؛ برای همین باید همیشه با EXPLAIN (ANALYZE, BUFFERS) واقعی تصمیم گرفت، نه فقط با نگاه کردن به لیست ایندکس ها. (سناریوهای 3، 5 و 7 هم همین ایندکس GIN را برای همین شرط دارند، اما چون شرط دیگری در کوئری شان گزینش پذیری بیشتری اضافه می کند، این اثر منفی آن جا دیده نشد.)

۵.۵ نمونه ی واقعی خروجی EXPLAIN ANALYZE

خروجی کامل EXPLAIN (ANALYZE, BUFFERS, SETTINGS) هر 8 سناریو - هم قبل و هم بعد از ایندکس گذاری، یعنی 16 فایل - در [results/explain/](#) ذخیره شده است. سه نمونه ی زیر مستقیماً از همان فایل هاست، فقط برای

جاشدن در صفحه خلاصه شده‌اند: گره‌های سطح-کارگر و برآورد هزینه‌ی برنامه‌ریز حذف شده‌اند، ولی زمان‌های واقعی و بافرها دست‌نخورده‌اند.
سناریوی 1، قبل و بعد - همان رگرسیون که در بالا توضیح داده شد:

Listing 9: results/explain/scenario_01.txt – BEFORE (excerpt)

```

1 Sort (actual time=4347.848..4379.101 rows=27 loops=1)
2   Buffers: shared hit=12087 read=249303
3   -> Finalize GroupAggregate (actual time=4347.418..4378.968 rows=27 loops=1)
4     -> Gather Merge (Workers Launched: 4)
5       -> Parallel Hash Join (actual time=2356.150..4191.300 rows=69766
        loops=5)
6         -> Parallel Seq Scan on title_ratings tr
7           (actual time=0.611..1577.639 rows=340413 loops=5)
8         -> Parallel Seq Scan on title_basics tb
9           Filter: (title_type = 'movie'::text)
10          Rows Removed by Filter: 2387111
11          (actual time=0.596..2089.660 rows=150775 loops=5)
12 Planning Time: 8.385 ms
13 Execution Time: 4381.153 ms

```

Listing 10: results/explain/scenario_01.after.txt – AFTER (excerpt)

```

1 Sort (actual time=8907.907..8948.027 rows=27 loops=1)
2   Buffers: shared hit=69029 read=136532 written=3892
3   I/O Timings: shared read=24943.451 write=416.247
4   -> Finalize GroupAggregate (actual time=8907.699..8947.988 rows=27 loops=1)
5     -> Gather Merge (Workers Launched: 4)
6       -> Parallel Hash Join (actual time=6890.573..8659.886 rows=69766
        loops=5)
7         -> Parallel Seq Scan on title_ratings tr
8           (actual time=0.224..1494.175 rows=340413 loops=5)
9         -> Parallel Bitmap Heap Scan on title_basics tb
10          Recheck Cond: (title_type = 'movie'::text)
11          Heap Blocks: exact=34108
12          (actual time=659.714..6046.800 rows=150775 loops=5)
13          -> Bitmap Index Scan on idx_title_basics_type_genres_gin
14            (actual time=588.927..588.927 rows=753874 loops=1)
15 Planning Time: 23.851 ms
16 Execution Time: 8953.258 ms

```

سناریوی 2، بعد از ایندکس - بزرگ‌ترین بهبود (از 93 به 12.7 میلی‌ثانیه):

Listing 11: results/explain/scenario_02.after.txt (excerpt)

```

1 Limit (actual time=2.379..12.610 rows=10 loops=1)
2   Buffers: shared hit=43 read=31
3   -> Nested Loop (actual time=2.377..12.596 rows=10 loops=1)
4     -> Index Scan using idx_title_ratings_votes_desc on title_ratings tr
5       (actual time=1.171..1.235 rows=12 loops=1)
6     -> Index Scan using title_basics_pkey on title_basics tb
7       Filter: (title_type = 'movie'::text)
8       (actual time=0.939..0.939 rows=1 loops=12)
9 Planning Time: 7.632 ms
10 Execution Time: 12.683 ms

```

سناریوی 8، قبل و بعد - افت بافر خوانده‌شده از 115,045 به 410:

Listing 12: results/explain/scenario_08.txt – BEFORE, main node only (excerpt)

```

1 Parallel Seq Scan on title_basics tb (actual time=2.135..687.571 rows=95056 loops
  =5)
2   Filter: ((runtime_minutes IS NOT NULL) AND (title_type = 'movie'::text))
3   Rows Removed by Filter: 2442830
4   Buffers: shared hit=115350 read=115044
5   -- (full plan: Nested Loop -> Sort -> Gather Merge -> WindowAgg above this node)
6   Execution Time: 2803.599 ms

```

Listing 13: results/explain/scenario_08.after.txt (excerpt)

```

1 Subquery Scan on ranked (actual time=1079.154..1927.038 rows=135 loops=1)
2   Buffers: shared hit=142987 read=410
3   -> WindowAgg (Run Condition: (row_number() OVER (?) <= 5))
4     -> Nested Loop (actual time=146.206..477.905 rows=148312 loops=5)
5       -> Parallel Bitmap Heap Scan on title_basics tb
6         Recheck Cond: ((title_type = 'movie'::text) AND
7           (runtime_minutes IS NOT NULL))
8         (actual time=146.106..382.596 rows=95056 loops=5)
9       -> Bitmap Index Scan on idx_title_basics_movie_runtime
10         (actual time=91.626..91.626 rows=475280 loops=1)
11 Planning Time: 5.891 ms
12 Execution Time: 1930.801 ms

```

۶ مباحث نظری

۱.۶ پارامترهای تنظیم PostgreSQL

در کنار مستندات رسمی، دو منبع دیگر هم برای تحقیق درباره‌ی این پارامترها مفیدند: postgresqlco.nf [۶] که توضیح و مقدار پیش‌فرض هر پارامتر را کنار هم نشان می‌دهد، و PgTune [۷] که با گرفتن مشخصات سخت‌افزار و نوع بار کاری، یک نقطه‌ی شروع برای این مقادیر پیشنهاد می‌دهد.

shared_buffers. این پارامتر اندازه‌ی کش مشترک پیج‌های پایگاه‌داده را در حافظه‌ی خود PostgreSQL تعیین می‌کند [۵]. بزرگ کردن زیاد آن همیشه بهتر نیست: چون سیستم‌عامل هم پیج‌ها را در کش خودش نگه می‌دارد، عددی بین 25% تا 40% حافظه‌ی کل معمولاً خوب است. بیشتر از این، همان داده دوبار (یک‌بار در کش PostgreSQL، یک‌بار در کش سیستم‌عامل) نگه داشته می‌شود بدون فایده‌ی اضافه.

work_mem. برخلاف shared_buffers، این مقدار برای هر عملیات Sort یا Hash در هر کوئری جداگانه است، نه یک‌بار برای کل سیستم. یک کوئری پیچیده با چند Hash/Sort می‌تواند چند برابر این مقدار حافظه مصرف کند. پس مقداری که برای یک اتصال خوب است، ممکن است برای صدها اتصال هم‌زمان کل حافظه‌ی سرور را تمام کند. در این پروژه، مقدار سراسری پایین نگه داشته شده و فقط در سطح نشست برای کوئری‌های سنگین بالا می‌رود.

effective_cache_size. این پارامتر هیچ حافظه‌ای تخصیص نمی‌دهد؛ فقط یک تخمین به برنامه‌ریز می‌دهد از این که چقدر داده احتمالاً در کش جا می‌شود. عدد بزرگ‌تر باعث می‌شود برنامه‌ریز هزینه‌ی I/O تصادفی را کمتر تخمین بزند و بیشتر به سمت ایندکس‌ها برود.

wal_compression و **default_toast_compression**. هر دو یک مبادله بین CPU و I/O دیسک هستند. lz4 سریع‌تر از الگوریتم پیش‌فرض pglz است و روی حجم کاری بارگذاری انبوه این پروژه، هزینه CPU اضافی آن خیلی کمتر از فایده‌ی کاهش نوشتار دیسک است.

autovacuum. PostgreSQL از کنترل هم‌روندی چندنسخه‌ای (MVCC) استفاده می‌کند: هر DELETE/UPDATE نسخه‌ی قدیمی سطر را پاک نمی‌کند، فقط آن را «مرده» علامت می‌زند تا تراکنش‌های دیگر که هنوز به آن نیاز دارند خراب نشوند. autovacuum این سطرهای مرده را پاک می‌کند. آستانه‌ی پیش‌فرض آن (درصدی از اندازه‌ی جدول) برای یک صف کار که خیلی سریع‌تر از رشد جدول، سطر مرده تولید می‌کند مناسب نیست؛ به همین دلیل جدول message_queue با آستانه‌ی مطلق (تعداد ثابت سطر) تنظیم شده، نه آستانه‌ی نسبی.

maintenance_work_mem. این پارامتر هم مثل work_mem یک محدودیت حافظه است، ولی نه برای کوئری‌های معمولی؛ برای عملیات‌های نگه‌داری مثل CREATE INDEX و VACUUM، ALTER TABLE استفاده می‌شود. چون این عملیات‌ها معمولاً یکی یکی و کم‌تعداد اجرا می‌شوند، نه هم‌زمان برای صدها اتصال مثل work_mem، می‌شود مقدار آن را خیلی بزرگ‌تر گذاشت بدون نگرانی از تمام‌شدن حافظه‌ی سرور. مقدار پیش‌فرض PostgreSQL فقط 64MB است؛ برای پروژه‌ای که باید ایندکس یک جدول 90 میلیون سطری بسازد، این عدد کم است — عدد بزرگ‌تر یعنی مرتب‌سازی ساخت ایندکس در حافظه انجام شود، نه با چند پاس روی دیسک. به همین دلیل در این پروژه 768MB گذاشته شده.

wal_buffers. قبل از این که رکوردهای WAL (لاگ پیش‌نوشت، برای بازیابی بعد از کرش) واقعاً روی دیسک نوشته شوند، اول در یک بافر مشترک در حافظه جمع می‌شوند؛ این پارامتر اندازه‌ی همان بافر است. مقدار پیش‌فرض آن 1- است، یعنی خودکار تنظیم شود (معمولاً حدود 1/32 از shared_buffers). بزرگ‌ترکردن آن به تراکنش‌های هم‌زمان زیاد کمک می‌کند، چون رکوردهای بیشتری قبل از یک نوشتار واقعی جمع می‌شوند؛ ولی بعد از یک حد مشخص (اندازه‌ی چیزی که بین دو چک‌پوینت واقعاً نوشته می‌شود) بزرگ‌ترکردن بیشتر فایده‌ای ندارد. در این پروژه روی 16MB تنظیم شده.

min_wal_size و **max_wal_size**. این دو، فضایی که PostgreSQL برای فایل‌های WAL نگه می‌دارد را کنترل می‌کنند و مستقیماً روی زمان‌بندی چک‌پوینت اثر می‌گذارند. رسیدن حجم WAL به max_wal_size باعث می‌شود یک چک‌پوینت زودتر از موعد اجرا شود. max_wal_size کوچک یعنی چک‌پوینت‌های مکرر — هزینه‌ی I/O بیشتر ولی بازیابی سریع‌تر بعد از کرش؛ max_wal_size بزرگ یعنی چک‌پوینت‌های کم‌تر — برای بارگذاری انبوه بهتر است، ولی بازیابی بعد از کرش کندتر می‌شود. مقدار پیش‌فرض PostgreSQL 16 برای این دو به‌ترتیب 80MB و 1GB است. در این پروژه، عمداً یک max_wal_size نسبتاً کوچک (4GB) انتخاب شده تا خط پایه رفتار واقعی یک بارگذاری I/O-محدود را نشان دهد.

seq_page_cost و **random_page_cost**. این دو، واحد هزینه‌ای هستند که برنامه‌ریز کوئری برای مقایسه‌ی پلن‌های مختلف استفاده می‌کند: seq_page_cost هزینه‌ی خواندن یک صفحه به‌صورت پی‌درپی است (پیش‌فرض 1.0)، و random_page_cost هزینه‌ی خواندن یک صفحه به‌صورت تصادفی (پیش‌فرض 4.0). این نسبت 4 به 1 از دنیای دیسک‌های مکانیکی می‌آید، جایی که یک جهش تصادفی سر دیسک واقعاً چند برابر کندتر از خواندن پی‌درپی بود. روی NVMe/SSD این تفاوت خیلی کمتر است، پس نگه‌داشتن random_page_cost روی 4 باعث می‌شود برنامه‌ریز به‌اشتباه فکر کند ایندکس همیشه گران‌تر از اسکن کامل است. در این پروژه random_page_cost روی 1.1 تنظیم شده تا با واقعیت سخت‌افزار سازگار باشد.

enable_seqscan، **enable_indexscan** و **enable_bitmapscan**. این سه، کلید قطع‌ووصل سطح-نشست هستند؛ خاموش کردن هرکدام آن روش دسترسی را کاملاً ممنوع نمی‌کند، فقط هزینه‌ی آن را مصنوعی خیلی بالا می‌برد تا برنامه‌ریز تقریباً همیشه از آن فرار کند. کاربرد اصلی‌شان تست و مقایسه است، نه تنظیم دائمی در postgresql.conf: مثلاً می‌شود موقتاً enable_bitmapscan را خاموش کرد تا ببینیم اگر ایندکس در دسترس نبود پلن چه شکلی می‌شد، بدون این که واقعاً ایندکس را حذف کنیم.

ما همین کار را واقعاً برای سناریوی 1 (بخش ۵) انجام دادیم: با `enable_bitmapscan = off` و `enable_indexscan` = off، برنامه‌ریز مجبور شد به جای Bitmap Heap Scan (که خودش انتخاب کرده بود، در 8953.3 میلی ثانیه) به Parallel Seq Scan برگردد - این بار در 14522.6 میلی ثانیه، یعنی حتی کندتر. نکته‌ی مهم: این عدد را نباید مستقیماً با عدد اصلی «قبل از ایندکس» (4381.2 میلی ثانیه) مقایسه کرد، چون آن اندازه‌گیری ساعت‌ها زودتر و با کش تازه‌تری از دیسک انجام شده بود؛ اما مقایسه‌ی این دو عدد در همین نشست (هر دو با وضعیت کش یکسان) نشان می‌دهد که برنامه‌ریز، با توجه به کش فعلی، واقعاً بهترین گزینه‌ی موجود را انتخاب کرده بود - فقط آن گزینه از عدد خط پایه‌ی اصلی بدتر بود.

۲.۶ مقایسه‌ی انواع ایندکس

- **B-Tree** (پیش‌فرض PostgreSQL): یک ساختار درختی که کلیدها را مرتب نگه می‌دارد. برای تساوی، بازه (<، >) و مرتب‌سازی خوب است. فقط ستون‌های اول یک ایندکس چندستونی قابل استفاده‌اند.
- **Hash**: فقط برای تساوی (=) خوب است؛ نه بازه و نه مرتب‌سازی را پشتیبانی نمی‌کند. از PostgreSQL 10 به بعد بعضی وقت‌ها کوچک‌تر و سریع‌تر از B-Tree است، اما این مزیت به‌ندرت به‌اندازه‌ای است که ارزش از دست‌دادن بازه/مرتب‌سازی را داشته باشد - به همین دلیل در این پروژه استفاده نشده.
- **GiST** (Generalized Search Tree): برخلاف بقیه که برای یک نوع داده‌ی خاص طراحی شده‌اند، GiST یک چارچوب عمومی است. با نوشتن چندتابع خاص (تشخیص هم‌پوشانی، فشردگی، ادغام)، می‌شود آن را برای داده‌های خیلی متفاوت به کار برد: داده‌ی هندسی/جغرافیایی (مثلاً در PostgreSQL)، بازه‌ها (range types)، جست‌وجوی نزدیک‌ترین همسایه (KNN)، با عملگر <->، و حتی جست‌وجوی متن به‌عنوان جایگزین GIN. ساخت و به‌روزرسانی آن معمولاً سریع‌تر از GIN است، ولی چون بعضی از پیاده‌سازی‌هایش lossy هستند (ممکن است نتیجه‌ی اضافی هم برگردد که باید دوباره چک شود)، جست‌وجو در آن کمی کندتر و اندازه‌اش بزرگ‌تر از یک GIN هم‌ارز است.
- **GIN** (Generalized Inverted Index): برای مقادیری که هر سطر چندتا از آن‌ها دارد - آرایه، JSONB، جست‌وجوی متن. برای هر مقدار داخلی، یک لیست از سطری که آن مقدار را دارند نگه می‌دارد. برای «آیا این مقدار توی این ستون هست؟» (>@، &&) خیلی سریع است، اما مفهوم ترتیب ندارد و فقط با Bitmap Index Scan در دسترس است، نه Index Scan ساده. با پسوند btree_gin، یک GIN می‌تواند چند ستون از نوع‌های مختلف را با هم ترکیب کند و - برخلاف B-Tree - با فیلتر روی هر کدام از این ستون‌ها به‌تنهایی هم کار کند.
- **BRIN** (Block Range Index): برای ستون‌هایی که ترتیب فیزیکی‌شان روی دیسک با ترتیب مقدارشان همخوانی دارد (مثلاً یک ستون زمانی که به‌ترتیب درج شده) خیلی کوچک و کم‌هزینه است، چون فقط بازه‌ی مقدار هر بلوک دیسک را نگه می‌دارد، نه هر سطر را. در این پروژه استفاده نشده چون داده‌ی IMDb بعد از عبور از صف پیام، دیگر لزوماً با همان ترتیبی که BRIN لازم دارد درج نمی‌شود.

کدام ایندکس برای JSONB مناسب است؟ از این چهار نوع، GIN انتخاب استاندارد PostgreSQL برای JSONB است - همان‌طور که بالا گفته شد، دقیقاً برای سؤال «آیا این کلید/مقدار توی این JSONB هست؟» (>@، ?) طراحی شده. GiST هم روی JSONB قابل استفاده است ولی رایج نیست و معمولاً کندتر از GIN عمل می‌کند. B-Tree و Hash اصلاً مناسب این کار نیستند، چون فقط تساوی/ترتیب کل مقدار JSONB را می‌فهمند، نه محتوای داخلش را.

۳.۶ JSONB و مکانیزم TOAST

تفاوت JSON و JSONB. نوع JSON متن خام ورودی را دقیقاً همان‌طور که نوشته شده نگه می‌دارد - با همان فاصله‌ها، همان ترتیب کلیدها، و حتی کلیدهای تکراری. هر بار که این مقدار خوانده یا پرس‌وجو شود، PostgreSQL باید کل متن را از نو تجزیه (parse) کند. JSONB برعکس این است: در زمان نوشتن یک‌بار تجزیه و به یک قالب باینری آماده تبدیل می‌شود. نتیجه یک مبادله‌ی ساده است: درج JSONB کمی کندتر است (چون باید ساخته شود)، ولی خواندن و جست‌وجو خیلی سریع‌تر است (چون تجزیه‌ی دوباره لازم نیست) - برای همین تقریباً همیشه JSONB به JSON ترجیح داده می‌شود.

JSONB چگونه در داخل PostgreSQL ذخیره می‌شود؟ مقدار JSONB به صورت یک قالب باینری از قبل تجزیه شده و «تجزیه شده» (decomposed binary format) ذخیره می‌شود، نه به صورت متن. این قالب یک هدر به همراه آرایه‌ای از افسس‌ها/طول‌ها دارد که مستقیماً به محل هر کلید و مقدار در همان مقدار باینری اشاره می‌کند. کلیدهای هر شیء بر اساس طول و سپس ترتیب باینری مرتب نگه داشته می‌شوند، پس پیدا کردن یک کلید مشخص با جست‌وجوی دودویی (binary search) روی همین آرایه ممکن است – بدون این‌که کل مقدار از اول خوانده شود. سه چیز در این فرایند تبدیل از بین می‌رود: فاصله‌های خالی متن اصلی، ترتیب اصلی کلیدها (چون بر اساس طول/بایت دوباره مرتب می‌شوند)، و کلیدهای تکراری (اگر یک کلید دوبار در ورودی بیاید، فقط آخرین مقدارش نگه داشته می‌شود). اعداد هم به جای متن، در یک نوع عددی داخلی ذخیره می‌شوند. همین ساختار باینری با افسس مستقیم است که هم جست‌وجوی سریع را ممکن می‌کند و هم پایه‌ی ساخت ایندکس GIN روی JSONB است.

عملگرها و توابع مهم JSONB. چندتای پرکاربردش: > یک فیلد/عضو را به صورت jsonb برمی‌گرداند؛ «- همان را به صورت متن برمی‌گرداند؛ @> می‌پرسد «آیا سمت چپ شامل سمت راست است؟» – همان عملگر containment که ایندکس GIN برایش ساخته می‌شود؛ ؟ می‌پرسد «آیا این کلید در سطح بالای JSONB وجود دارد؟». از توابع هم jsonb_set() (تغییر یک مقدار داخلی بدون خواندن-تغییر-نوشتن کل شیء در کد برنامه)، jsonb_agg() (جمع کردن چند سطر در یک آرایه‌ی JSONB) و jsonb_each() (باز کردن یک JSONB به سطری‌های کلید-مقدار) پرکاربردند. در این پروژه، ستون title_ratings.metadata با عملگر @> قابل پرس‌وجوست – مثلاً: 'metadata @> {"popular": true}' که روی داده‌ی واقعی این پروژه 17,906 سطر برمی‌گرداند.

TOAST (The Oversized-Attribute Storage Technique) [۲] مکانیزمی است که به PostgreSQL اجازه می‌دهد مقادیر متغیر طول بزرگ (مثل JSONB، متن یا آرایه) را بیرون از سطر اصلی، در یک جدول کمکی نگه دارد. چهار روش ذخیره برای هر ستون وجود دارد: PLAIN (بدون فشرده‌سازی یا انتقال؛ برای انواع با اندازه‌ی ثابت)، MAIN (فشرده‌سازی درون‌خطی، انتقال به بیرون فقط اگر واقعاً لازم باشد)، EXTENDED (پیش‌فرض JSONB؛ هم فشرده‌سازی هم انتقال به بیرون مجاز است) و EXTERNAL (بدون فشرده‌سازی ولی انتقال به بیرون مجاز – برای وقتی که دسترسی تصادفی به بخشی از مقدار مهم‌تر از فضااست). فعال‌شدن این مکانیزم – همان‌طور که در بخش ۳ گفتیم – بر اساس کل سطر است، نه هر ستون؛ همین نکته پایه‌ی طراحی بسته‌بندی دسته‌ای این پروژه است.

ساخت ایندکس GIN روی JSONB. دستور ساده است: CREATE INDEX ... ON table USING GIN (jsonb_column). حالت پیش‌فرض (jsonb_ops) هم کلیدها هم مقدارها را ایندکس می‌کند و از @>، @، ?، |، ?& پشتیبانی می‌کند. یک گزینه‌ی دیگر، jsonb_path_ops، فقط از @> پشتیبانی می‌کند ولی ایندکس کوچک‌تر و سریع‌تری می‌سازد – برای ستونی مثل metadata در این پروژه که (اگر روزی لازم شود) فقط قرار است با containment پرس‌وجو شود، jsonb_path_ops انتخاب بهتری از پیش‌فرض بود.

JSONB یا جدول نرمال شده؟ قاعده‌ی ساده: اگر قرار است روی یک فیلد داخلی زیاد فیلتر/پیوند/گروه‌بندی کنید، بهتر است آن فیلد یک ستون واقعی با ایندکس B-Tree باشد – برنامه‌ریز آمار بهتری از یک ستون واقعی دارد تا از داخل یک JSONB. اما اگر ساختار داده متغیر است (هر سطر ممکن است کلیدهای متفاوتی داشته باشد)، یا قرار است کل شیء با هم خوانده/نوشته شود، JSONB بهتر است چون از ساختن چندین جدول کمکی با ستون‌های اغلب NULL جلوگیری می‌کند. ستون metadata در این پروژه دقیقاً نمونه‌ی دوم است: داده‌ی مشتق‌شده‌ای که هیچ‌کدام از هشت سناریو روی آن فیلتر نمی‌کند، پس نیازی به نرمال‌سازی ندارد.

۴.۶ نظریه‌ی صف پیام

الگوی Competing Consumers. در این الگو، چند مصرف‌کننده‌ی مستقل از یک صف مشترک کار برمی‌دارند تا پردازش سریع‌تر شود. مشکل اصلی این است که یک کار نباید توسط بیش از یک مصرف‌کننده برداشته شود، بدون این‌که مصرف‌کننده‌ها مجبور شوند پشت سر هم صف بکشند.

مکانیزم NOTIFY/LISTEN. این دو دستور، مکانیزم داخلی PostgreSQL برای اطلاع‌رسانی بین نشست‌ها هستند. یک نشست با LISTEN channel_name روی یک کانال ثبت‌نام می‌کند؛ هر نشست دیگری با NOTIFY chan-nel_name, 'payload' (یا تابع pg_notify) یک پیام کوتاه متنی به همه‌ی نشست‌های ثبت‌نام‌شده روی آن کانال می‌فرستد — این پیام معمولاً بعد از commit همان تراکنش، به‌صورت غیرهم‌زمان به گیرنده‌ها می‌رسد.

چرا NOTIFY/LISTEN به‌تنهایی برای صف کافی نیست. مشکل اصلی این است که NOTIFY هیچ حافظه‌ای ندارد: اگر لحظه‌ی ارسال هیچ نشستی روی آن کانال گوش نمی‌داد، آن پیام برای همیشه گم می‌شود — نه صف‌بندی، نه امکان بازپخش. همچنین NOTIFY به‌صورت پخش (broadcast) به همه‌ی گوش‌دهنده‌ها می‌رسد، نه فقط یکی؛ یعنی خودش هیچ تضمینی نمی‌دهد که یک پیام دقیقاً یک‌بار توسط یکی از مصرف‌کننده‌ها برداشته شود — دقیقاً همان مشکلی که Competing Consumers باید حلش کند. به همین دلیل این پروژه، حالت واقعی صف (وضعیت، مهلت، شمارنده‌ی تلاش) را در جدول message_queue نگه می‌دارد، نه در NOTIFY. اگر لازم شود، می‌شود NOTIFY/LISTEN را فقط به‌عنوان یک زنگ‌خطر سبک اضافه کرد — مصرف‌کننده به‌جای polling دوره‌ای منتظر NOTIFY بماند — ولی وضعیت واقعی هر پیام همچنان باید در جدول بماند.

SKIP LOCKED در برابر قفل معمولی. یک SELECT ... FOR UPDATE می‌کند، اگر سطرهای قبلاً قفل شده باشند، منتظر می‌ماند تا آزاد شوند — یعنی مصرف‌کننده‌ها پشت سر هم صف می‌کشند. اضافه کردن SKIP LOCKED [۴] این رفتار را عوض می‌کند: هر سطر قفل‌شده از نتیجه حذف می‌شود و تراکنش می‌رود سراغ سطر آزاد بعدی. نتیجه این است که N مصرف‌کننده‌ی هم‌زمان، N دسته‌ی کاملاً جدا از صف برمی‌دارند، بدون این‌که منتظر هم بمانند — دقیقاً همان چیزی که Competing Consumers نیاز دارد.

تضمین‌های تحویل. سه سطح رایج وجود دارد: at-most-once (پیام ممکن است هیچ‌وقت پردازش نشود، ولی هیچ‌وقت هم دوبار پردازش نمی‌شود)، at-least-once (پیام حتماً حداقل یک‌بار پردازش می‌شود، ولی ممکن است در بعضی حالت‌ها — مثلاً خرابی مصرف‌کننده قبل از تأیید — بیش از یک‌بار هم پردازش شود)، و exactly-once (که در سیستم‌های توزیع‌شده واقعی، بدون یک لایه‌ی هماهنگ‌کننده‌ی اضافه، تقریباً غیرممکن یا خیلی گران است). صف این پروژه تضمین at-least-once می‌دهد: مکانیزم مهلت با locked_until تضمین می‌کند اگر مصرف‌کننده‌ای قبل از ack از کار بیفتد، پیام (بعد از تمام شدن مهلت و اجرای reaper_expired) دوباره به صف برمی‌گردد — ولی این یعنی ممکن است همان پیام دوبار پردازش شود.

بی‌اثری (Idempotency). چون تضمین at-least-once اجازه می‌دهد پردازش تکراری اتفاق بیفتد، مصرف‌کننده باید طوری طراحی شود که پردازش دوباره‌ی یک پیام، نتیجه‌ی نهایی را عوض نکند. در این پروژه، این کار با ON CONFLICT DO NOTHING روی کلید اصلی هر جدول انجام شده: اگر یک دسته دوبار درج شود (مثلاً به‌خاطر برگشتن به صف)، تلاش دوم سطرهای تکراری را نادیده می‌گیرد، بدون خطا و بدون تغییر داده.

جدول-به‌عنوان-صف در برابر راه‌حل تخصصی. همان‌طور که پروژه‌ی Postgres For Everything [۸] هم نشان می‌دهد، PostgreSQL می‌تواند نقش خیلی از ابزارهای تخصصی — از جمله صف پیام — را بازی کند. استفاده از یک جدول ساده به‌جای یک message broker تخصصی (مثل RabbitMQ یا Kafka) چند مزیت دارد: هیچ زیرساخت جدیدی لازم نیست، و از همه مهم‌تر، می‌شود enqueue کردن یک پیام و تغییر بقیه‌ی داده‌ها را در یک تراکنش واحد انجام داد — یک broker جدا این هماهنگی رایگان را نمی‌دهد. عیبش این است که زیر بار خیلی سنگین (هزاران تولیدکننده/مصرف‌کننده هم‌زمان) به‌اندازه‌ی یک broker تخصصی مقیاس‌پذیر نیست، و هر بار poll یک کوئری واقعی روی همان پایگاه‌داده‌ای است که به کارهای دیگر هم سرویس می‌دهد؛ ویژگی‌های پیشرفته‌تر (مسیریابی پیچیده، بازپخش از یک نقطه‌ی مشخص، داشبورد نظارتی) هم باید از صفر ساخته شوند. برای حجم کار این پروژه — یک بارگذاری انبوه، نه پیام‌رسانی بین چند سرویس مستقل — سادگی جدول-به‌عنوان-صف کاملاً کافی و حتی ترجیح داده می‌شود.

۷ نتیجه‌گیری

این پروژه یک معماری کامل برای بارگذاری و کوئری گرفتن از مجموعه داده‌ی IMDb روی PostgreSQL 16 ساخت: یک صف پیام با `SELECT ... FOR UPDATE SKIP LOCKED` برای بارگذاری هم‌زمان و بدون رقابت، بسته‌بندی دسته‌ای برای رد کردن آستانه‌ی فشرده‌سازی TOAST، هشت سناریوی کوئری طبق سند پروژه، و شش ایندکس که هرکدام به یک شرط مشخص در این سناریوها وصل است. هر بخش از این پروژه – اسکیمای پایگاه داده، توابع صف، تولیدکننده و مصرف‌کننده، کوئری‌ها، و طراحی ایندکس – حداقل یک‌بار با یک بازبینی چندعاملی بررسی شده و باگ‌های واقعی پیداشده در آن اصلاح شده‌اند. برای بخش‌هایی که نیازی به اجرای واقعی نداشتند (نحو SQL، منطق Python، تست‌های خودکار)، درستی با ابزارهایی مثل `py_compile` و آزمون دستی چک شده است.

۱.۷ محدودیت این نگارش

همه‌ی عددهای بخش ۵ از یک اجرای واقعی PostgreSQL 16 داخل Docker به دست آمده‌اند، نه از پیش‌بینی. تنها محدودیت واقعی این اجرا فضای دیسک بود: لپ‌تاپی که این پروژه رویش اجرا شد فقط حدود 30 گیگابایت فضای خالی داشت، و بیشتر از این هم نمی‌شد جا باز کرد. دیتاست کامل IMDb – مخصوصاً جدول `title_principals` با نزدیک 90 میلیون سطر – توی این فضا جا نمی‌شد. برای همین فقط همین یک جدول با فلگ `limit-rows` – به 8 میلیون سطر (حدود 9%) محدود شد؛ شش جدول دیگر کامل بارگذاری شدند (جزئیات در بخش ۳.۵). نتیجه‌ی عملی این محدودیت این است که سناریوی 5 – تنها سناریویی که از `title_principals` می‌خواند – روی حجمی کوچک‌تر از واقعیت سنجیده شده؛ بهبود واقعی آن روی دیتاست کامل احتمالاً بیشتر از عدد گزارش شده است. یک نتیجه‌ی دیگر هم ارزش گفتن دارد: ایندکس GIN ترکیبی، سناریوی 1 را کندتر کرد نه سریع‌تر. این عدد را حذف یا نرم نکردیم، چون خودش یک درس واقعی از این پروژه است. ایندکس‌گذاری فقط با نگاه کردن به لیست شرط‌های کوئری کافی نیست. باید با `EXPLAIN (ANALYZE, BUFFERS)` واقعی سنجید، چون گزینش‌پذیری کم یک شرط – مثل `title_type = 'movie'` – می‌تواند نتیجه‌ی برعکس بدهد.

۲.۷ کارهای آینده

اگر بخواهیم این پروژه را گسترش دهیم، سه مسیر طبیعی برای ادامه وجود دارد: افزایش کردن (partitioning) جدول `title_principals` بر اساس `category` برای کم کردن بیشتر حجم اسکن‌شده در سناریوی 5؛ استفاده از دیدهای مادی شده (materialized views) برای سناریوهایی که به‌طور دوره‌ای اجرا می‌شوند نه بلادرنگ؛ و پایش مستمر عملکرد با `pg_stat_statements` برای پیدا کردن خودکار کوئری‌هایی که با تغییر داده در طول زمان، به ایندکس یا تنظیم جدیدی نیاز پیدا می‌کنند.

مراجع

[۱] PostgreSQL Global Development Group، «PostgreSQL 16 Documentation»، <https://www.postgresql.org/docs/16/index.html>

[۲] PostgreSQL Global Development Group، «TOAST (The Oversized-Attribute Storage Technique)»، <https://www.postgresql.org/docs/16/storage-toast.html>

[۳] PostgreSQL Global Development Group، «GIN Indexes»، <https://www.postgresql.org/docs/16/gin.html>

[۴] PostgreSQL Global Development Group، «SELECT ... FOR UPDATE SKIP LOCKED»، <https://www.postgresql.org/docs/16/sql-select.html#SQL-FOR-UPDATE-SKIP-LOCKED>

Resource Consumption» «PostgreSQL Global Development Group [Δ]
[https://www.postgresql.org/docs/16/«\(shared_buffers, work_mem, ...\) runtime-config-resource.html](https://www.postgresql.org/docs/16/«(shared_buffers, work_mem, ...) runtime-config-resource.html)

<https://postgresqlco. «PostgreSQL Configuration Documentation» «postgresqlco.nf nf/>

<https://pgtune.leopard.in.ua/> «PostgreSQL Configuration Calculator» «PgTune [Υ]

https://github.com/olshansk/ «Postgres For Everything» «Daniel Olshansky postgres_for_everything

<https://datasets.imdbws.com/> «IMDb Non-Commercial Datasets» «IMDb [ϑ]

<https://developer.imdb.com/ «Non-Commercial Datasets – Documentation» «IMDb \. non-commercial-datasets/>